Agentic AI: Job Application Assistant

AI - IA II, Activity 1: Agentic AI System using Modern LLM-based Tools

Group members	[Name 1, Roll No.] / [Name 2, Roll No.] / [Name 3, Roll No.]
Lab batch	[Batch]
Course / Faculty	Artificial Intelligence / Dr. Ayesha Hakim
Stack	Python, LangChain / LangGraph, OpenRouter (LLM API), Flask (web UI), pypdf, python-docx

1. Introduction

A classical AI agent perceives its environment, decides, and acts using rules written by a programmer. **Agentic AI** keeps this perceive-decide-act idea but lets a large language model (LLM) make the decisions: the LLM receives a *goal*, plans steps, calls *tools*, reads the results and repeats until it can give a final answer. This project builds a small agent of this kind. No model is trained; the work is in tool design, orchestration, memory and an observable execution trace.

2. Problem Statement

Students applying for jobs rarely know how well their resume covers what a job description (JD) asks for. The task is to build an agent that analyses *any* resume uploaded by the user against *any* JD (uploaded or pasted), finds skill gaps, computes a simple match score and recommends honest improvements, through a simple web page.

3. Agent Goal

Goal: given a user-uploaded resume (PDF / DOCX / TXT) and a JD (file or pasted text), produce (a) a Resume-JD Skill Match Score, (b) matched and missing skills, (c) relevant experience, (d) the major gaps, (e) concrete resume improvements and learning areas, and (f) a saved report. The user can then ask follow-up questions.

4. Agent Role

A single **LLM orchestrator** (an OpenRouter model used through LangChain, driven by a LangGraph ReAct-style loop). It does not follow a fixed script: from the goal, the system prompt, the tool descriptions and the observations so far it decides *which tool to call next*, decides when enough information is available, writes the recommendations, and answers follow-ups. Its limits are fixed by the system prompt (use tool outputs as facts, never invent experience, no hiring predictions).

5. Tools

Tool	What it does	Why it is needed
load_documents	Reads the user-uploaded resume and JD (.pdf / .docx / .txt) from the paths given in the goal, once, and stores them in working memory.	The LLM cannot open files; also avoids passing long text through the prompt.
analyze_skill_gap	Matches resume against JD skills (lexicon, word-boundary regex): required, matched, missing, preferred, relevant experience lines.	Deterministic, repeatable skill detection; the LLM reasons over its result.
calculate_match_score	Score = matched required skills / required skills x 100, plus a band (limited / moderate / strong overlap).	Arithmetic is delegated to code - LLMs can miscalculate.

Tool	What it does	Why it is needed
generate_report	Builds the structured markdown report from working memory and the advice written by the LLM; saves it.	Gives a consistent, complete deliverable.

All tools return JSON with a status and a short summary, and never raise exceptions: failures come back as error observations the agent can explain. No tool reads a fixed file: the file paths come from the goal message, which the web UI builds from the user's uploads.

6. Memory

Conversation memory (short-term / contextual): a LangGraph `MemorySaver` checkpointer stores every message, tool call and observation under a thread id. When the user later asks 'What are my three biggest skill gaps?', the LLM sees the earlier analysis and answers without reloading files.
Working memory: a small `SESSION` dictionary shared by the tools holds the parsed documents, the gap analysis and the score, so each file is read once per analysis. In the web UI each browser session has its own agent and conversation thread, so follow-up questions refer to that user's own resume and JD. Memory lasts for the running process only; no database is used.

7. Planning Strategy

The agent uses a **ReAct-style** loop (Reason, Act, Observe). The system prompt describes the tools and the working rules but does *not* hard-code a call order; tool descriptions state prerequisites (for example, the score tool needs the gap analysis first). In a normal run the LLM chooses load, analyse, score, report. If a tool reports an error, the agent explains the problem instead of continuing; on follow-ups it answers from memory without any tool call.

8. System Architecture

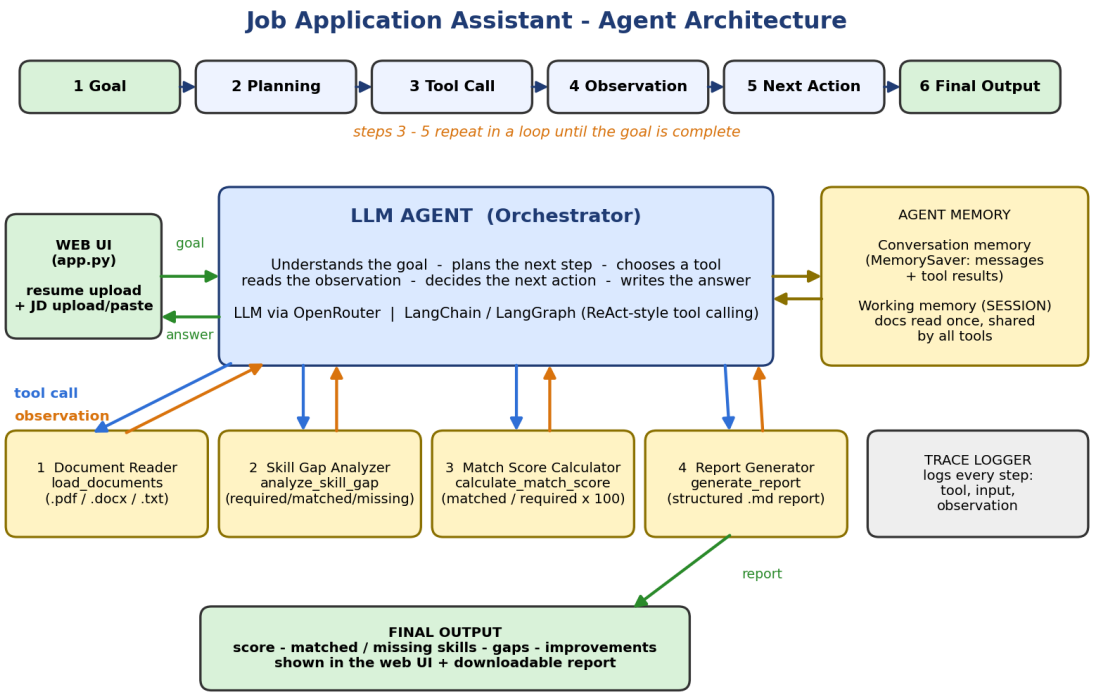


Figure 1: LLM orchestrator, four tools, conversation and working memory, trace logger.

9. Agent Workflow

Goal (user uploads resume + JD and clicks Analyze; the UI turns this into the goal message) → **Planning** (LLM decides it needs the documents first) → **Tool call** (load_documents) → **Observation** (documents loaded) → **Next action** (analyze_skill_gap, then calculate_match_score, then generate_report) → **Final output** (score, skills, gaps, advice, saved report). Steps 3-5 repeat until the goal is complete.

10. Implementation

File	Role
app.py	Flask web UI: upload resume (PDF/DOCX/TXT), upload or paste JD, Analyze button, results, execution trace, report download, follow-up chat. Saves uploads to a temporary folder, passes the paths to the agent, deletes them afterwards.
agent.py	Builds the OpenRouter LLM and the agent (create_agent + MemorySaver), AnalysisSession (one agent + memory thread per browser session), error handling, CLI (--resume, --jd, --ask, --offline).
tools.py	The four tools, the skill lexicon, safe_tool wrapper (turns exceptions into error observations), SESSION working memory.
agent_trace.py	LangChain callback handler: writes the execution trace to logs/ and keeps it as structured events that the UI displays.
offline_llm.py	Rule-based stand-in for the LLM, used only for offline testing.
tests/run_tests.py	Automated checks for tools, agent, memory, report, uploads and the web UI (Section 13).
data/, logs/, outputs/, diagrams/, report/	Demo/test inputs (the samples are used only by the demo button, the CLI default and tests), traces, generated reports, diagram, this report.

11. Agent Interaction Logs

The trace records the goal, each step's tool selected, tool input and observation, and the final response. It deliberately logs *actions and observations*, not the model's hidden reasoning.

About this trace: it was produced for a *non-sample* resume and JD (a web-developer resume, data/example_resume_webdev.txt, against a Frontend Developer JD) with the **--offline** rule-based stand-in model, because the build environment had no OpenRouter access. A live run writes the same format to logs/agent_trace.txt, and the web UI shows the same steps as a checklist (Goal received, Documents loaded, Skill gaps analyzed, Match score calculated, Report generated, Final response prepared) with the tool name of each step. To refresh this section with a live trace, replace logs/example_trace.txt and run report/make_report.py.

```
======================================================================
AGENT SESSION  2026-10-03 18:50:36   LLM: OFFLINE rule-based stand-in (not a real LLM)
======================================================================

Goal received:
  Analyze my resume at 'data/example_resume_webdev.txt' against the job description at 'data/example_jd_frontend.txt'.
                    Tell me how well they match and how I can improve.

Step 1:
  Agent decision : I need the resume and job description before any analysis.
  Tool selected  : load_documents
  Tool input     : {"resume_path": "data/example_resume_webdev.txt", "jd_path": "data/example_jd_frontend.txt"}
  Observation    : Resume (61 words) and job description (36 words) loaded into working memory.

Step 2:
  Agent decision : Documents are loaded; next I compare skills.
  Tool selected  : analyze_skill_gap
  Tool input     : {}
  Observation    : 8 required skills identified: 6 matched, 2 missing (Docker, AWS). Preferred: 0/2 matched.

Step 3:
  Agent decision : Gap analysis is done; next I compute the score.
  Tool selected  : calculate_match_score
  Tool input     : {}
  Observation    : Resume-JD Skill Match Score: 75% (6 of 8 required skills; strong overlap).
```

```
Step 4:
  Agent decision : I have the score; now I write the improvement report.
  Tool selected  : generate_report
  Tool input     : {"recommendations": ["Add a project or coursework bullet that shows hands-on Docker.", "Add a
                   project or coursework bullet that shows hands-on AWS."], "learning_areas": ["Docker: take an
                   introductory tutorial, then use i
  Observation    : Report generated and saved to outputs/improvement_report.md


Final response to user:
  Analysis complete. The report was saved to outputs/improvement_report.md.
  [full report text omitted here - see outputs/example_report.md]

(tools used this turn: load_documents, analyze_skill_gap, calculate_match_score, generate_report)

Follow-up received:
  What are my three biggest skill gaps?


Final response to user:
  From the analysis I already did (no need to re-read the files), your biggest gaps are: Docker, AWS. They are listed
                   in the order the job description mentions them.

(tools used this turn: none - answered from memory)
```

12. Web UI, Sample Inputs and Outputs

UI. The page (python app.py) has: Upload Resume (PDF / DOCX / TXT); Job Description as an uploaded file *or* pasted text; an **Analyze Resume** button; and a 'Try with sample data' button for demos. After the agent finishes, the page shows the match score, matched and missing skills, experience analysis, major gaps, resume improvement recommendations, suggested learning areas, a summary, the agent execution trace and a Download report button, followed by a follow-up question box that uses the same agent memory.

Input (example): data/example_resume_webdev.txt (web developer: JavaScript, React, HTML, CSS, Node.js, Git, REST APIs) and data/example_jd_frontend.txt (Frontend Developer, 8 required and 2 preferred skills). **Output** (generated report, abridged):

```
# Resume-JD Analysis Report
## Resume-JD Skill Match Score
**75%** (6 of 8 required skills matched - strong overlap).
This is a keyword-overlap indicator for learning purposes, not a hiring prediction.
## Matched Skills
- JavaScript
- React
- HTML
- CSS
- Node.js
- REST APIs
## Missing Skills
- Docker
- AWS
Preferred skills matched: none  |  missing: Kubernetes, CI/CD
## Relevant Experience
- Online Shop UI: built with React and CSS, consuming REST APIs.  _(shows: REST APIs, React, CSS)_
- Portfolio website using HTML and CSS.  _(shows: HTML, CSS)_
- Web Developer Intern (3 months): built React components and fixed UI bugs using Git.  _(shows: React)_
## Major Gaps
Top gaps, in the order the job description lists them:
- Docker
- AWS
## Recommended Resume Improvements
- Add a project or coursework bullet that shows hands-on Docker.
- Add a project or coursework bullet that shows hands-on AWS.
## Suggested Learning Areas
- Docker: take an introductory tutorial, then use it in a small project.
- AWS: take an introductory tutorial, then use it in a small project.
- Kubernetes: take an introductory tutorial, then use it in a small project.
- CI/CD: take an introductory tutorial, then use it in a small project.
## Overall Summary
Your resume covers 6 of 8 required skills. The main gaps are Docker, AWS. Closing these with honest, project-backed evidence is
```

Required skills: 8; matched: 6; missing: 2 (Docker, AWS). Skill Match Score = 6/8 = **75%**. A different resume or JD gives a different result (for example, a web resume against a DevOps JD gives 0%); this is covered by the tests. The score is a keyword-overlap indicator, not a hiring probability; preferred skills

are reported separately and are not part of the score.

13. Testing

Run with `python tests/run_tests.py` using the offline stand-in model. This verifies the tools, memory, tracing, error handling, file uploads (TXT / PDF / DOCX), pasted JDs, missing-input handling and the web UI routes; it does **not** measure the reasoning quality of a real LLM.

Result	Test scenario
PASS	1. Normal analysis (4 tools, in order, report saved, score 70%)
PASS	3. Follow-up answered from memory (no tools re-run)
PASS	3b. Second follow-up recalls score
PASS	2. Missing skills detected on DevOps JD (score 25%)
PASS	4a. Missing resume file -> clear error, no report, no crash
PASS	4b. CLI with missing file -> exit code != 0, message, no traceback
PASS	4c. Empty resume file -> clear error
PASS	4d. No files given -> agent asks for them, calls no tools
PASS	5. Tool failure handled gracefully (error explained, run stops cleanly)
PASS	6. Tool called before prerequisites -> error message, not exception
PASS	7. PDF resume is read correctly
PASS	8. Live mode without API key -> friendly message, no traceback
PASS	9. UI: TXT resume + pasted JD -> analysis (score 83%, missing Docker)
PASS	9b. UI: execution trace has goal -> 4 tool steps -> final, with tool names
PASS	9c. UI: recommendations, learning areas and summary come back for display
PASS	10. UI follow-up answered from memory about the uploaded files (no tools re-run)
PASS	10b. Follow-up before any analysis -> clear message
PASS	11. Report downloads and contains the uploaded resume's content, not sample data
PASS	12. Different resume + JD -> different output (DevOps: 100%, nothing missing)
PASS	12b. Same JD, other resume -> score changes (web resume vs DevOps JD: 0%, 5+ missing)
PASS	12c. Demo fallback still works (sample files: 70%)
PASS	12d. New analysis replaces the old report (no stale content)
PASS	13. DOCX resume (incl. table text) is read: Docker, Statistics, Data Visualization matched
PASS	13b. PDF resume + pasted JD works
PASS	14a. No resume -> 400 with message
PASS	14b. No JD -> 400 with message
PASS	14c. JD as file AND text -> 400 (ambiguous)
PASS	14d. Unsupported file type -> 400
PASS	14e. Empty resume file -> 400, no agent run
PASS	14f. Corrupt PDF -> 400 friendly message
PASS	14g. JD with no recognisable skills -> agent reports problem, no fake score
PASS	15. Live mode without API key -> 503 with setup message (no traceback)
PASS	15b. Uploaded files are deleted after the run
PASS	16. tools.py / offline_llm.py never reference the sample files

34/34 checks passed

The most important check is that the system is not tied to the sample files: tests 12 and 12b analyse different resume/JD pairs and require different scores and missing-skill lists, test 11 requires the downloaded report to contain the uploaded resume's text, and test 16 checks that the tools never reference the sample files.

14. Classical vs Modern Agent Comparison

Classical Agent (Syllabus)	Modern Agent (This Activity)	How it appears in this project
Rule-based	LLM-driven	The LLM decides which tool to call; only the skill matcher is rule-based, inside a tool.
Static logic	Dynamic planning	The call order is not hard-coded; errors and follow-ups change what the agent does next.
No memory	Contextual memory	MemorySaver keeps the conversation; SESSION shares data between tools.
Single-step	Multi-step reasoning	Four chained tool calls, each using the previous observation.

A classical simple-reflex agent for the same task would map percepts to actions with fixed IF-THEN rules (for example, 'IF a required skill word is absent THEN flag it') and could not take a new goal, remember an earlier result or explain a failure. The modern agent is more flexible but less predictable and needs a trace to be trusted. This project is a **hybrid**: reliable rule-based logic sits inside the tools, the LLM orchestrates them.

15. Analysis and Reflection

What worked. Small tools with clear descriptions and prerequisites make orchestration easy to follow. Returning a one-line summary from every tool made the trace readable. Sharing data through working memory removed redundant file reads and kept long text out of the prompt.

Tool orchestration and reliability. The agent depends on the LLM choosing correct tool calls; free OpenRouter models differ in tool-calling quality and rate limits, so a run can fail or vary. Failures are caught and reported (no raw traceback). Calculation is delegated to a tool for reliability.

Limitations. Skill detection is keyword-based with a lexicon of about 35 skills (synonyms outside it are missed; it cannot judge depth or seniority). The score is a simple ratio, not a real ATS or hiring result. Scanned PDFs are unsupported. Memory is lost when the program exits. The agent is not fully autonomous: it works with four predefined tools inside limits set by the prompt. The resume text is sent to the LLM provider through the API. The web UI is a single-user local app: analyses run one at a time and the tools share one working memory. Uploaded files are deleted after each analysis; the generated report and trace log stay on disk.

Future improvements. Semantic (embedding) matching of skills, a reviewer agent that checks the recommendations, persistent memory, support for more job roles, multi-user support.

16. Conclusion

The project extends the classical goal-based agent into an agentic system: through a simple web UI, a user uploads any resume and JD; an LLM receives the goal, chooses among four tools, observes the results, keeps context across turns and produces a structured, goal-oriented output, with every step visible in a trace. It meets the activity requirements of one LLM agent, at least two tools, multi-step reasoning and observable behaviour.